

คู่มือปฏิบัติการ · รายวิชา 5534408 การพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่

LabStock เฟส 2

เชื่อมต่อฐานข้อมูล MySQL ผ่าน REST API (PHP)

ต่อจากเฟส 1 (Step 0–11) · Step 12–17 · ประมาณ 5 คาบ

เปลี่ยนข้อมูลจำลองในหน่วยความจำให้เป็นฐานข้อมูลจริง โดยหน้าจอแทบไม่ต้องแก้

Flutter 3.35 · PHP 8 · MySQL/MariaDB (XAMPP) · โค้ดทั้งหมดที่ <https://github.com/kittipongnuanyai/App1> · ปรับปรุง 30 ก.ย. 2569

สำหรับผู้สอน: เตรียมอะไร และสอนอย่างไร

สิ่งที่ต้องเตรียมก่อนคาบแรกของเฟส 2

- 1. push โค้ดขึ้น GitHub พร้อม tag **phase1-ui** และ **phase2-mysql** แล้วทดสอบเปิด <https://github.com/kittipongnuanyai/App1/compare/phase1-ui...phase2-mysql> ว่าเห็น diff
- 2. ทุกเครื่องในห้องมี XAMPP และสตาร์ท Apache + MySQL ได้ (ทดสอบก่อนคาบ 1 วัน เพราะปัญหาพอร์ตชนกันนาน)
- 3. เครื่องผู้สอน: import labstock.sql และวาง api ไว้แล้ว เปิด index.php ได้ทันทีตอนสาธิต
- 4. เตรียม IP ของเครื่องผู้สอนไว้ ให้นักศึกษาที่ XAMPP มีปัญหาชี้แอปมาที่เซิร์ฟเวอร์ของผู้สอนชั่วคราว (ทุกคนอยู่วง Wi-Fi เดียวกัน) จะได้ไม่หยุดเรียน
- 5. ติดตั้ง Thunder Client (VS Code) หรือใช้ curl เพื่อสาธิต POST/PUT
- 6. แจ้งล่วงหน้าว่า**ไม่ใช่ InfinityFree** (ดูเหตุผลใน Step 17) ถ้าจะให้ส่งงานบนโฮสต์ ให้สมัคร AlwaysData ไว้ก่อน

แผนการสอน

Step	หัวข้อ	เป้าหมาย	นาที
12	ตั้งต้นที่จุดเดียวกัน: ดึงโค้ดเฟส 1 จาก GitHub	ทุกคนมีโค้ดเฟส 1 ที่รันได้เหมือนกัน และเปิด XAMPP (Apache + MySQL) ได้	20
13	สร้างฐานข้อมูล MySQL	มีฐานข้อมูล labstock 3 ตาราง (users, items, borrows) พร้อมข้อมูลเริ่มต้น และอธิบายความสัมพันธ์ได้	40
14	สร้าง REST API ด้วย PHP	วาง PHP 7 ไฟล์ใน htdocs แล้วเรียกผ่านเบราว์เซอร์/curl ได้ JSON ครบทุก endpoint	70
15	ฝั่ง Flutter: ApiService และการแปลง JSON	แอปมีชั้น ApiService ที่เรียก API ได้ทุก endpoint และโมเดลแปลง JSON ↔ object ได้	60
16	เปลี่ยน AppState ไปใช้ API และปรับหน้าจอ	แอปทำงานกับข้อมูลจริงใน MySQL ครบทุก flow: login, รายการ, ยืม, คืน, เพิ่ม/แก้ไข, สมัครสมาชิก	80
17	นำขึ้นโฮสต์จริง และก้าวต่อไป	เข้าใจข้อจำกัดของโฮสต์ฟรี นำ API ขึ้นโฮสต์ที่รองรับแอปมือถือได้ และเห็นภาพเฟส 3	40
	รวม		310

ลำดับการสอนที่แนะนำในแต่ละ Step

1. **เห็นก่อน (5 นาที)** ผู้สอนสาธิตผลลัพธ์ปลายทางของ Step นั้น เช่น curl แล้วเห็นแถวเพิ่มใน phpMyAdmin
2. **เข้าใจ (10–15 นาที)** อธิบายกล่อง "ความรู้ใหม่" ด้วยโค้ดจริงบนจอ ไม่ต้องอธิบายทุกบรรทัด เน้นบรรทัดที่เป็นแนวคิด
3. **ทำเอง (ส่วนใหญ่ของเวลา)** นักศึกษาคัดลอกไฟล์ใหม่จาก GitHub และแก้ไฟล์เดิมตาม diff — ห้ามพิมพ์ตามทั้งไฟล์ เสียเวลาและผิดง่าย
4. **Checkpoint (5 นาที)** เดินดูทีละโต๊ะตามรายการในกล่องเขียว ใครผ่านให้ทำท้าทาย ใครไม่ผ่านให้จับคู่กับคนที่ผ่าน
5. **จุดที่ต้องรอทั้งห้อง:** จบ Step 14 (API ต้องได้ทุกคน) และจบ Step 16 รอบแรก (login + list) ก่อนไปต่อ

วิธีให้นักศึกษา "โหลดโค้ดเป็นส่วน ๆ"

โค้ดใน GitHub แยก commit ตาม Step (Step 13, 14, 15, 16) นักศึกษาเลือกได้ 3 ระดับตามความพร้อม:

ระดับ	วิธี	เหมาะกับ
A ทำเอง	อ่าน diff ในคู่มือ แล้วแก้โค้ดตัวเองทีละบรรทัด	คนที่เฟส 1 ทำเองสำเร็จ
B คัดลอกทีละไฟล์	เปิดไฟล์บน GitHub → ปุ่ม Raw → คัดลอกทั้งไฟล์ทับของเดิม (เฉพาะไฟล์ที่ Step นั้นบอก)	คนที่ตามทันแต่พิมพ์ช้า
C ข้ามไปจุดจบของ Step	<code>git checkout <commit ของ Step></code> หรือดาวน์โหลด ZIP ของ tag phase2-mysql แล้วแก้ค่า kApiBase / config.php อย่างเดียว	คนที่ตามไม่ทัน ให้ไปต่อพร้อมเพื่อนก่อน แล้วค่อยย้อนอ่าน

ปัญหาที่พบบ่อยและวิธีแก้

อาการ	สาเหตุ	แก้
แอปขึ้น "เชื่อมต่อเซิร์ฟเวอร์ไม่ได้" บน Android Emulator	ใช้ localhost ซึ่งหมายถึงตัว emulator เอง	ใช้ 10.0.2.2 (api_config.dart ทำให้อัตโนมัติ ตรวจสอบว่า kApiBase ยังเป็น localhost)
มือถือจริงต่อไม่ได้ แต่ emulator ได้	Firewall ของ Windows บล็อก Apache หรือมือถือคนละ Wi-Fi	อนุญาต Apache ใน Firewall · เช็ก IP ด้วยเบราว์เซอร์มือถือก่อน
Android ขึ้น CLEARTEXT communication not permitted	Android 9+ ห้าม http	เพิ่ม usesCleartextTraffic="true" ใน AndroidManifest (Step 15)
iOS ขึ้น App Transport Security	iOS ห้าม http	เพิ่ม NSAllowsArbitraryLoads ใน Info.plist (Step 15)
"เซิร์ฟเวอร์ตอบไม่ใช่ JSON"	PHP error (HTML) · path ผิด (404) · หรือโฮสต์ฟรีมีระบบ	เปิด URL เดียวกันในเบราว์เซอร์ดูข้อความจริง

	ตรวจเบราว์เซอร์	
ภาษาไทยใน JSON เป็น ??? หรือ à,	ฐานข้อมูลไม่ใช่ utf8mb4 หรือ อ่าน res.body	import ด้วยไฟล์ที่ให้ (utf8mb4) · ใช้ utf8.decode(res.bodyBytes)
MySQL ใน XAMPP สตาร์ทไม่ขึ้น	พอร์ต 3306 ถูกใช้แล้ว	ปิด MySQL service ตัวอื่น หรือเปลี่ยนพอร์ตและแก้ DB_PORT
Login ได้แต่รายการว่าง	items.php error หรือ category ในตารางไม่ตรง enum	เปิด items.php ในเบราว์เซอร์ · category ต้องเป็น board/sensor/cable/module/tool
ยืมแล้วจำนวนไม่ลด	ลืม await refresh() หรือ PUT/POST ไปไม่ถึง	ดู log Apache หรือ print ใน _send

ตั้งต้นที่จุดเดียวกัน: ดึงโค้ดเฟส 1 จาก GitHub

เวลาโดยประมาณ 20 นาที

เป้าหมาย: ทุกคนมีโค้ดเฟส 1 ที่รันได้เหมือนกัน และเปิด XAMPP (Apache + MySQL) ได้

คนที่ทำเฟส 1 ไม่ทัน หรือโค้ดมีปัญหา ให้ใช้โค้ดจาก GitHub แทนของตัวเองได้เลย รีบมี tag ชื่อ `phase1-ui` คือจุดจบของเฟส 1 พอดี

วิธีที่ 1: ดาวน์โหลด ZIP (ไม่ต้องใช้ git)

1. เปิด <https://github.com/kittipongnuanyai/App1/releases/tag/phase1-ui> หรือกดปุ่ม Code → Download ZIP
2. แยกไฟล์ เปลี่ยนชื่อโฟลเดอร์เป็น `labstock`
3. เปิดโฟลเดอร์ใน VS Code แล้วรัน `flutter pub get` และ `flutter run`

วิธีที่ 2: ใช้ git (แนะนำ จะได้ดึงโค้ดขึ้นถัดไปได้ง่าย)

```
git clone https://github.com/kittipongnuanyai/App1.git labstock
cd labstock
git checkout phase1-ui      # ไปยังจุดจบเฟส 1
flutter pub get
flutter run
```

ระหว่างทำเฟส 2 ถ้าอยากดูโค้ดของชั้นไหน ให้เปิดหน้า **compare** บน GitHub จะเห็นว่าแต่ละไฟล์ต้องแก้บรรทัดไหน (สีเขียว = เพิ่ม, สีแดง = ลบ):

```
https://github.com/kittipongnuanyai/App1/compare/phase1-ui...phase2-mysql
```

เปิด XAMPP

1. เปิด XAMPP Control Panel → Start **Apache** และ **MySQL** (ทั้งสองต้องเป็นสีเขียว)
2. เปิดเบราว์เซอร์ไปที่ `http://localhost/phpmyadmin` ต้องเห็นหน้า phpMyAdmin
3. โฟลเดอร์เว็บของ XAMPP คือ `C:\xampp\htdocs` (Windows) หรือ `/Applications/XAMPP/htdocs` (Mac)

✓ Checkpoint

- แอปเฟส 1 รันได้ login ด้วยชื่ออะไรก็ได้ เห็นการดอปกรณ์ 8 ใบ
- phpMyAdmin เปิดได้
- รู้ว่าโฟลเดอร์ htdocs อยู่ที่ไหน

สำหรับผู้สอน

- MySQL ใน XAMPP สตาร์ทไม่ขึ้น มักเพราะพอร์ต 3306 ถูกใช้โดย MySQL ตัวอื่นที่ติดตั้งไว้ตอนเรียนวิชาฐานข้อมูล ให้ปิด service นั้นก่อน หรือเปลี่ยนพอร์ตใน my.ini แล้วแก้ DB_PORT ใน config.php ตาม
- Apache ไม่ขึ้นเพราะพอร์ต 80 ชนกับ Skype/IIS ให้เปลี่ยนเป็น 8080 แล้วแก้ URL ในแอปเป็น `http://localhost:8080/labstock/api`
- ให้นักศึกษาที่ทำเฟส 1 เองสำเร็จ ยังคงใช้โค้ดตัวเองได้ แต่ถ้าติดตอน Step 16 ให้สลับมาใช้โค้ดจาก GitHub ทันที อย่าเสียเวลาดีบั๊กของเก่า

สร้างฐานข้อมูล MySQL

เวลาโดยประมาณ 40 นาที

เป้าหมาย: มีฐานข้อมูล labstock 3 ตาราง (users, items, borrows) พร้อมข้อมูลเริ่มต้น และอธิบายความสัมพันธ์ได้

โมเดล 3 ตัวที่เขียนใน Dart ตอนเฟส 1 (AppUser, Equipment, Borrow) ตอนนี้จะกลายเป็นตาราง 3 ตาราง คอลัมน์ตั้งชื่อแบบ snake_case ตามธรรมเนียม SQL (`student_id`) ส่วน Dart ใช้ camelCase (`studentId`) ตอนแปลง JSON จะต้องจับคู่กัน

ความรู้ที่ทบทวนจากวิชาฐานข้อมูล

- PRIMARY KEY + AUTO_INCREMENT
- FOREIGN KEY (borrows → users, items)
- ENUM สำหรับค่าที่มีตัวเลือกจำกัด (role, category)
- utf8mb4 เพื่อเก็บภาษาไทยและอีโมจิ
- ไม่เก็บรหัสผ่านจริง เก็บ `password_hash` ที่ PHP สร้างด้วย bcrypt
- DATE_ADD(CURDATE(), ...) สร้างข้อมูลทดสอบที่สัมพันธ์กับวันนี้

1) นำเข้าไฟล์ SQL

1. ดาวน์โหลด `server/labstock.sql` จาก GitHub
2. phpMyAdmin → แท็บ **Import** → เลือกไฟล์ → กด Go (ไฟล์มีคำสั่ง CREATE DATABASE ให้แล้ว)
3. คู่มือ ต้องมีฐานข้อมูล **labstock** และตาราง users / items / borrows

2) โครงสร้างและข้อมูลเริ่มต้น

server/labstock.sql

ไฟล์ใหม่ · คัดลอกทั้งไฟล์

```
-- =====
-- LabStock - โครงสร้างฐานข้อมูล + ข้อมูลเริ่มต้น
-- วิธีใช้: phpMyAdmin → เลือกฐานข้อมูล labstock → Import ไฟล์นี้
-- (บนโฮสต์ฟรี ให้สร้างฐานข้อมูลจากแผงควบคุมก่อน แล้ว Import โดยไม่ต้อง CREATE DATABASE)
-- =====

CREATE DATABASE IF NOT EXISTS labstock CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE labstock;

-- ----- ผู้ใช้ -----
```

```

DROP TABLE IF EXISTS borrows;
DROP TABLE IF EXISTS items;
DROP TABLE IF EXISTS users;

CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  username    VARCHAR(50) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,          -- เก็บ hash ไม่เก็บรหัสผ่านจริง
  name        VARCHAR(100) NOT NULL,
  student_id  VARCHAR(20) NOT NULL,
  role        ENUM('student','admin') NOT NULL DEFAULT 'student',
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----- อุปกรณ์ -----
CREATE TABLE items (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  category    ENUM('board','sensor','cable','module','tool') NOT NULL,  -- ตรงกับ en
um ในแอป
  total       INT NOT NULL DEFAULT 0,
  available   INT NOT NULL DEFAULT 0,          -- คงเหลือ (ลดเมื่อยืม เพิ่มเมื่อคืน)
  description TEXT,
  image_url   VARCHAR(255) DEFAULT NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ----- การยืม -----
CREATE TABLE borrows (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT NOT NULL,
  item_id     INT NOT NULL,
  quantity    INT NOT NULL DEFAULT 1,
  borrow_date DATE NOT NULL,
  due_date    DATE NOT NULL,
  return_date DATE DEFAULT NULL,          -- NULL = ยังไม่คืน
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (item_id) REFERENCES items(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- ข้อมูลเริ่มต้น (รหัสผ่านของทุกคนคือ 1234)
-- =====

INSERT INTO users (username, password_hash, name, student_id, role) VALUES
('admin', '$2y$10$lhGUPq0kvHtXKdBIx37ln.UP8f1/on70F6GtNp4iugqrclaFpcKJq', 'อ.สมหญิง
แล็บดี', 'ICE-ADMIN', 'admin'),
('somchai', '$2y$10$lhGUPq0kvHtXKdBIx37ln.UP8f1/on70F6GtNp4iugqrclaFpcKJq', 'สมชาย ใ
จดี', '674659001', 'student'),
('somsri', '$2y$10$lhGUPq0kvHtXKdBIx37ln.UP8f1/on70F6GtNp4iugqrclaFpcKJq', 'สมศรี มีสุ
ข', '674659002', 'student');

INSERT INTO items (name, category, total, available, description) VALUES
('Arduino UNO R3', 'board', 10, 5, 'บอร์ดไมโครคอนโทรลเลอร์ ATmega328P ใช้สำหรับ
เรียนพื้นฐาน Embedded และ IoT รองรับการพัฒนาโปรแกรมผ่าน Arduino IDE'),
('DHT22', 'sensor', 6, 0, 'เซนเซอร์วัดอุณหภูมิและความชื้นแบบดิจิทัล ความแม่นยำ
สูงกว่า DHT11'),

```

```
( 'สาย USB Type-B',          'cable', 15, 12, 'สาย USB A-to-B ยาว 1 เมตร สำหรับต่อบอร์ด
Arduino UNO'),
( 'Servo SG90',              'module', 8, 3, 'เซอร์โวมอเตอร์ขนาดเล็ก หมุนได้ 0-180 องศา
แรงบิด 1.8 kg·cm'),
( 'ESP32 DevKit',           'board', 12, 7, 'บอร์ดไมโครคอนโทรลเลอร์ที่มี Wi-Fi และ Blue
tooth ในตัว เหมาะกับงาน IoT'),
( 'Ultrasonic HC-SR04',     'sensor', 10, 9, 'เซนเซอร์วัดระยะทางด้วยคลื่นอัลตราโซนิก ระยะ
2-400 ซม.'),
( 'สาย Jumper (ชุด 40 เส้น)', 'cable', 20, 14, 'สายจัมเปอร์ ผู้-ผู้ ยาว 20 ซม. ชุดละ 40 เส้น'),
( 'มัลติมิเตอร์ดิจิทัล',   'tool', 5, 2, 'เครื่องวัดแรงดัน กระแส และความต้านทาน แบบดิจิทัล');

-- การยืมของ somchai (id=2): ใช้วันที่สัมพันธ์กับวันนี้ เพื่อให้ป้าย "ใกล้ครบกำหนด" แสดงถูกต้อง
INSERT INTO borrows (user_id, item_id, quantity, borrow_date, due_date, return_date)
VALUES
(2, 1, 1, DATE_SUB(CURDATE(), INTERVAL 5 DAY), DATE_ADD(CURDATE(), INTERVAL 2 DAY),
NULL),
(2, 4, 2, DATE_SUB(CURDATE(), INTERVAL 2 DAY), DATE_ADD(CURDATE(), INTERVAL 5 DAY),
NULL),
(2, 3, 1, DATE_SUB(CURDATE(), INTERVAL 20 DAY), DATE_SUB(CURDATE(), INTERVAL 13 DA
Y), DATE_SUB(CURDATE(), INTERVAL 14 DAY)),
(2, 6, 1, DATE_SUB(CURDATE(), INTERVAL 30 DAY), DATE_SUB(CURDATE(), INTERVAL 23 DA
Y), DATE_SUB(CURDATE(), INTERVAL 23 DAY));
```

3) ความสัมพันธ์

ER แบบย่อ

7 บรรทัด

```
users (1) —< borrows (n) >— (1) items
  id          user_id    id
  username    item_id    name
  password_hash quantity  category
  name        borrow_date total
  student_id  due_date    available ← ลดเมื่อยืม เพิ่มเมื่อคืน
  role        return_date description (NULL = ยังไม่คืน)
```

✓ Checkpoint

- phpMyAdmin → labstock → users มี 3 แถว (admin, somchai, somsri)
- items มี 8 แถว · borrows มี 4 แถว
- ลองรัน SQL: `SELECT b.*, i.name FROM borrows b JOIN items i ON i.id=b.item_id WHERE return_date IS NULL` ต้องได้ 2 แถว
- ตอบได้: ทำไม available ต้องเป็นคอลัมน์แยกจาก total

ท้าทาย

- เพิ่มนักศึกษาอีก 1 คนด้วย INSERT (ใช้ password_hash ตัวเดียวกับ somchai ก็ได้ = รหัส 1234)
- เขียน SQL หาว่าอุปกรณ์ชิ้นไหนถูกยืมมากที่สุด

สร้าง REST API ด้วย PHP

เวลาโดยประมาณ 70 นาที

เป้าหมาย: วาง PHP 7 ไฟล์ใน htdocs แล้วเรียกผ่านเบราว์เซอร์/curl ได้ JSON ครบทุก endpoint

แอปมือถือไม่ควรต่อ MySQL ตรง ๆ (เปิดพอร์ตฐานข้อมูลสู่อินเทอร์เน็ตอันตราย และโฮสต์ส่วนใหญ่ไม่อนุญาต) จึงต้องมี **ตัวกลาง** คือ PHP ที่รับคำขอ HTTP → ค่อยกับ MySQL → ตอบกลับเป็น JSON แอปเรียก PHP ผ่าน URL เหมือนเปิดเว็บ

ความรู้ใหม่ในขั้นนี้

- REST: ใช้ HTTP method บอกความหมาย GET = อ่าน · POST = สร้าง · PUT = แก้ไข
- รูปแบบคำตอบเดียวกันทุก endpoint: `{"ok":true,"data":...}` หรือ `{"ok":false,"error":"..."}`
- PDO + prepared statement (เครื่องหมาย ?) ป้องกัน SQL injection
- `password_hash` / `password_verify`
- Transaction (`beginTransaction` / `commit` / `rollBack`) สำหรับการยืมที่ต้องแก้ 2 ตารางพร้อมกัน
- header CORS และ Content-Type: application/json

1) วางไฟล์

สร้างโฟลเดอร์ `htdocs/labstock/api` แล้วคัดลอก 7 ไฟล์จาก `server/api/` บน GitHub ไปไว้ทั้งหมด จากนั้นเปิด `http://localhost/labstock/api/index.php` ต้องเห็น JSON ที่นับจำนวนแล้วได้

โครงสร้างบนเซิร์ฟเวอร์

10 บรรทัด

```
htdocs/
└─ labstock/
   └─ api/
      ├── config.php    ← ค่าเชื่อมต่อ (แก้ไฟล์นี้ไฟล์เดียวเมื่อย้ายโฮสต์)
      ├── db.php        ← เชื่อม PDO + ฟังก์ชัน json_ok / json_error / input
      ├── index.php     ← หน้าเช็คสถานะ
      ├── login.php     ← POST
      ├── register.php  ← POST
      ├── items.php     ← GET / POST / PUT
      └── borrows.php   ← GET / POST / PUT
```

2) config.php และ db.php (โครงสร้างพื้นฐาน)

`server/api/config.php`

ไฟล์ใหม่ · คัดลอกทั้งไฟล์

```

<?php
// =====
// LabStock API - ค่าเชื่อมต่อนฐานข้อมูล
// แก่ 4 ค่านี้ให้ตรงกับเครื่อง/โฮสต์ของตัวเอง แล้วไม่ต้องแก้ไฟล์อื่น
// =====

// --- XAMPP ในเครื่องตัวเอง (ค่าเริ่มต้น) ---
define('DB_HOST', '127.0.0.1');
define('DB_PORT', 3306); // XAMPP = 3306 · MAMP = 8889
define('DB_NAME', 'labstock');
define('DB_USER', 'root');
define('DB_PASS', '');

// --- ตัวอย่างค่าสำหรับโฮสต์ฟรี (AlwaysData) ---
// define('DB_HOST', 'mysql-ชื่อบัญชี.alwaysdata.net');
// define('DB_PORT', 3306);
// define('DB_NAME', 'ชื่อบัญชี_labstock');
// define('DB_USER', 'ชื่อบัญชี');
// define('DB_PASS', 'รหัสผ่านฐานข้อมูล');

```

server/api/db.php

ไฟล์ใหม่ · คัดลอกทั้งไฟล์

```

<?php
// =====
// LabStock API - เชื่อมต่อนฐานข้อมูล + ฟังก์ชันช่วยตอบ JSON
// ทุก endpoint จะ require_once ไฟล์นี้เป็นบรรทัดแรก
// =====
require_once __DIR__ . '/config.php';

// ----- ส่วนหัวของทุกคำตอบ -----
header('Content-Type: application/json; charset=utf-8');
header('Access-Control-Allow-Origin: *'); // ให้ Flutter web /
เครื่องอื่น เรียกได้
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') { http_response_code(204); exit; }

// ----- เชื่อมต่อด้วย PDO -----
try {
    $pdo = new PDO(
        'mysql:host=' . DB_HOST . ';port=' . DB_PORT . ';dbname=' . DB_NAME . ';char
set=utf8mb4',
        DB_USER,
        DB_PASS,
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, // ผิดพลาด → โยน
exception
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC, // ได้ array แบบ
key = ชื่อคอลัมน์
            PDO::ATTR_EMULATE_PREPARES => false,
        ]
    );
} catch (PDOException $e) {
    json_error('เชื่อมต่อนฐานข้อมูลไม่ได้: ' . $e->getMessage(), 500);
}

```

```
// ----- ฟังก์ชันช่วย -----

/** ตอบสำเร็จ: {"ok":true,"data":...} */
function json_ok($data = null): void {
    echo json_encode(['ok' => true, 'data' => $data], JSON_UNESCAPED_UNICODE);
    exit;
}

/** ตอบผิดพลาด: {"ok":false,"error":"..."} พร้อม HTTP status */
function json_error(string $message, int $status = 400): void {
    http_response_code($status);
    echo json_encode(['ok' => false, 'error' => $message], JSON_UNESCAPED_UNICODE);
    exit;
}

/** อ่านข้อมูลที่ส่งมา รองรับทั้ง JSON body และ form-data */
function input(): array {
    $raw = file_get_contents('php://input');
    $json = json_decode($raw, true);
    if (is_array($json)) return $json;
    return $_POST ?: [];
}

/** ดึงค่าจาก input โดยบังคับว่าต้องมี ไม่มี → ตอบ 400 */
function need(array $in, string $key) {
    if (!isset($in[$key]) || $in[$key] === '') json_error("ต้องส่งค่า '$key'");
    return $in[$key];
}

/** แปลงแถว users ให้ไม่ส่ง password_hash ออกไป */
function user_public(array $row): array {
    unset($row['password_hash']);
    $row['id'] = (int) $row['id'];
    return $row;
}

/** แปลงชนิดตัวเลขของแถว items ให้เป็น int (JSON จะได้ไม่เป็น string) */
function item_row(array $row): array {
    $row['id'] = (int) $row['id'];
    $row['total'] = (int) $row['total'];
    $row['available'] = (int) $row['available'];
    return $row;
}

function borrow_row(array $row): array {
    foreach (['id', 'user_id', 'item_id', 'quantity'] as $k) $row[$k] = (int) $row[$k];
    return $row;
}

```

สังเกต: ทุก endpoint จะเริ่มด้วย `require_once __DIR__ . '/db.php';` แล้วได้ตัวแปร `$pdo` กับฟังก์ชันช่วยมาใช้ทันที

3) index.php

```
<?php
// หน้าแรกของ API - เปิดในเบราว์เซอร์เพื่อเช็คเวิร์กและฐานข้อมูลพร้อม
require_once __DIR__ . '/db.php';

$count = [];
foreach (['users', 'items', 'borrows'] as $t) {
    $count[$t] = (int) $pdo->query("SELECT COUNT(*) FROM $t")->fetchColumn();
}

json_ok([
    'app'      => 'LabStock API',
    'version'  => '1.0',
    'php'      => PHP_VERSION,
    'database' => DB_NAME,
    'rows'     => $count,
    'endpoints' => [
        'POST login.php'      => '{username, password}',
        'POST register.php'   => '{username, password, name, student_id}',
        'GET items.php'       => 'รายการอุปกรณ์ทั้งหมด (หรือ ?id=)',
        'POST items.php'      => 'เพิ่มอุปกรณ์ {name, category, total, description}',
        'PUT items.php'       => 'แก้ไขอุปกรณ์ {id, name, category, total, description}',
        'GET borrows.php?user_id=' => 'รายการยืมของผู้ใช้',
        'POST borrows.php'     => 'ยืม {user_id, item_id, quantity, due_date}',
        'PUT borrows.php'     => 'คืน {id}',
    ],
]);
```

4) login.php และ register.php

```
<?php
// POST login.php {username, password} → ข้อมูลผู้ใช้ (ไม่มีรหัสผ่าน)
require_once __DIR__ . '/db.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') json_error('ต้องใช้ POST', 405);

$in      = input();
$username = trim($in['username']);
$password = $in['password'];

$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ?');
$stmt->execute([$username]);
$user = $stmt->fetch();

// password_verify เปรียบรหัสผ่านกับ hash ที่เก็บไว้ (ไม่เก็บรหัสผ่านจริงในฐานข้อมูล)
if (!$user || !password_verify($password, $user['password_hash'])) {
    json_error('ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง', 401);
}

json_ok(user_public($user));
```

```

<?php
// POST register.php {username, password, name, student_id} → ผู้ใช้ใหม่ (บทบาท student)
require_once __DIR__ . '/db.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') json_error('ต้องใช้ POST', 405);

$in          = input();
$username    = trim(need($in, 'username'));
$password    = need($in, 'password');
$name        = trim(need($in, 'name'));
$student_id  = trim(need($in, 'student_id'));

if (strlen($password) < 6) json_error('รหัสผ่านต้องไม่น้อยกว่า 6 ตัวอักษร');

// ตรวจสอบชื่อซ้ำ
$stmt = $pdo->prepare('SELECT id FROM users WHERE username = ?');
$stmt->execute([$username]);
if ($stmt->fetch()) json_error('ชื่อผู้ใช้นี้ถูกใช้แล้ว', 409);

$stmt = $pdo->prepare(
    'INSERT INTO users (username, password_hash, name, student_id, role) VALUES (?, ?, ?, ?, ?)'
);
$stmt->execute([$username, password_hash($password, PASSWORD_DEFAULT), $name, $student_id, 'student']);

$id = (int) $pdo->lastInsertId();
$user = $pdo->query("SELECT * FROM users WHERE id = $id")->fetch();
json_ok(user_public($user));

```

5) items.php

```

<?php
// items.php - อุปกรณ์
// GET items.php          → รายการทั้งหมด
// GET items.php?id=3     → ชื่นเดียว
// POST items.php         → เพิ่ม {name, category, total, description}
// PUT items.php          → แก้ไข {id, name, category, total, description}
require_once __DIR__ . '/db.php';

$method = $_SERVER['REQUEST_METHOD'];
$categories = ['board', 'sensor', 'cable', 'module', 'tool'];

// ----- GET -----
if ($method === 'GET') {
    if (isset($_GET['id'])) {
        $stmt = $pdo->prepare('SELECT * FROM items WHERE id = ?');
        $stmt->execute([(int) $_GET['id']]);
        $row = $stmt->fetch();
        if (!$row) json_error('ไม่พบอุปกรณ์', 404);
        json_ok(item_row($row));
    }
}

```

```

    }
    $rows = $pdo->query('SELECT * FROM items ORDER BY name')->fetchAll();
    json_ok(array_map('item_row', $rows));
}

// ----- POST / PUT ใช้ข้อมูลชุดเดียวกัน -----
$in          = input();
$name        = trim(need($in, 'name'));
$category    = need($in, 'category');
$total       = (int) need($in, 'total');
$description = trim($in['description'] ?? '');
$image_url   = $in['image_url'] ?? null;

if (!in_array($category, $categories, true)) json_error('หมวดหมู่ไม่ถูกต้อง');
if ($total <= 0) json_error('จำนวนต้องมากกว่า 0');

if ($method === 'POST') {
    $stmt = $pdo->prepare(
        'INSERT INTO items (name, category, total, available, description, image_url)
        VALUES (?, ?, ?, ?, ?, ?)'
    );
    $stmt->execute([$name, $category, $total, $total, $description, $image_url]);
    $id = (int) $pdo->lastInsertId();
    json_ok(item_row($pdo->query("SELECT * FROM items WHERE id = $id")->fetch()));
}

if ($method === 'PUT') {
    $id = (int) need($in, 'id');
    $stmt = $pdo->prepare('SELECT * FROM items WHERE id = ?');
    $stmt->execute([$id]);
    $old = $stmt->fetch();
    if (!$old) json_error('ไม่พบอุปกรณ์', 404);

    // จำนวนใหม่ต้องไม่น้อยกว่าที่ถูกยืมอยู่ แล้วคำนวณ available ใหม่
    $borrowed = (int) $old['total'] - (int) $old['available'];
    if ($total < $borrowed) json_error("จำนวนต้องไม่น้อยกว่าที่ถูกยืมอยู่ ($borrowed)");
    $available = $total - $borrowed;

    $stmt = $pdo->prepare(
        'UPDATE items SET name = ?, category = ?, total = ?, available = ?, description = ?, image_url = ?
        WHERE id = ?'
    );
    $stmt->execute([$name, $category, $total, $available, $description, $image_url, $id]);
    json_ok(item_row($pdo->query("SELECT * FROM items WHERE id = $id")->fetch()));
}

json_error('method ไม่รองรับ', 405);

```

6) borrows.php (มี Transaction)

server/api/borrows.php

ไฟล์ใหม่ · คัดลอกทั้งไฟล์

```

<?php
// borrows.php - การยืม/คืน
// GET borrows.php?user_id=2 → รายการยืมของผู้ใช้ (ทั้งกำลังยืมและคืนแล้ว)
// POST borrows.php → ยืม {user_id, item_id, quantity, due_date: 'YYYY-MM-DD'}
// PUT borrows.php → คืน {id}
require_once __DIR__ . '/db.php';

$method = $_SERVER['REQUEST_METHOD'];

// ----- GET -----
if ($method === 'GET') {
    $user_id = (int) ($_GET['user_id'] ?? 0);
    if ($user_id <= 0) json_error("ต้องส่ง ?user_id=");
    $stmt = $pdo->prepare(
        'SELECT b.*, i.name AS item_name
        FROM borrows b JOIN items i ON i.id = b.item_id
        WHERE b.user_id = ?
        ORDER BY b.return_date IS NOT NULL, b.due_date'
    );
    $stmt->execute([$user_id]);
    json_ok(array_map('borrow_row', $stmt->fetchAll()));
}

$in = input();

// ----- POST: ยืม -----
if ($method === 'POST') {
    $user_id = (int) need($in, 'user_id');
    $item_id = (int) need($in, 'item_id');
    $quantity = (int) need($in, 'quantity');
    $due_date = need($in, 'due_date');
    if ($quantity <= 0) json_error('จำนวนต้องมากกว่า 0');

    // Transaction: ดัดสต็อกและบันทึกการยืมต้องสำเร็จพร้อมกัน ไม่งั้นยกเลิกทั้งคู่
    $pdo->beginTransaction();
    try {
        $stmt = $pdo->prepare('SELECT available FROM items WHERE id = ? FOR UPDATE');
        $stmt->execute([$item_id]);
        $item = $stmt->fetch();
        if (!$item) throw new RuntimeException('ไม่พบอุปกรณ์');
        if ((int) $item['available'] < $quantity) throw new RuntimeException('จำนวนคงเหลือไม่พอ');

        $pdo->prepare('UPDATE items SET available = available - ? WHERE id = ?')
            ->execute([$quantity, $item_id]);
        $pdo->prepare(
            'INSERT INTO borrows (user_id, item_id, quantity, borrow_date, due_date)
            VALUES (?, ?, ?, CURDATE(), ?)'
        )->execute([$user_id, $item_id, $quantity, $due_date]);
        $id = (int) $pdo->lastInsertId();
        $pdo->commit();
    } catch (Exception $e) {
        $pdo->rollBack();
        json_error($e->getMessage());
    }
}

```

```

        $row = $pdo->query(
            "SELECT b.*, i.name AS item_name FROM borrows b JOIN items i ON i.id = b.item_id WHERE b.id = $id"
        )->fetch();
        json_ok(borrow_row($row));
    }

    // ----- PUT: คืน -----
    if ($method === 'PUT') {
        $id = (int) need($in, 'id');

        $pdo->beginTransaction();
        try {
            $stmt = $pdo->prepare('SELECT * FROM borrows WHERE id = ? FOR UPDATE');
            $stmt->execute([$id]);
            $b = $stmt->fetch();
            if (!$b) throw new RuntimeException('ไม่พบรายการยืม');
            if ($b['return_date'] !== null) throw new RuntimeException('รายการนี้คืนแล้ว');

            $pdo->prepare('UPDATE borrows SET return_date = CURDATE() WHERE id = ?')->execute([$id]);
            $pdo->prepare('UPDATE items SET available = LEAST(total, available + ?) WHERE id = ?')
                ->execute([(int) $b['quantity'], (int) $b['item_id']]);
            $pdo->commit();
        } catch (Exception $e) {
            $pdo->rollBack();
            json_error($e->getMessage());
        }

        $row = $pdo->query(
            "SELECT b.*, i.name AS item_name FROM borrows b JOIN items i ON i.id = b.item_id WHERE b.id = $id"
        )->fetch();
        json_ok(borrow_row($row));
    }

    json_error('method ไม่รองรับ', 405);

```

7) ทดสอบ

GET ทดสอบในเบราว์เซอร์ได้เลย ส่วน POST/PUT ใช้ curl (มีใน Windows 10+ และ Mac) หรือ extension Thunder Client ใน VS Code

ทดสอบ API

11 บรรทัด

เปิดในเบราว์เซอร์

http://localhost/labstock/api/index.php

http://localhost/labstock/api/items.php

http://localhost/labstock/api/borrows.php?user_id=2

curl (Windows: ใช้ PowerShell หรือ cmd)

curl -X POST http://localhost/labstock/api/login.php -H "Content-Type: application/json" -d '{"username":"somchai","password":"1234"}'

curl -X POST http://localhost/labstock/api/borrows.php -H "Content-Type: applicatio

```
n/json" -d '{"user_id":2,"item_id":1,"quantity":1,"due_date":"2026-10-15"}'
```

```
curl -X PUT http://localhost/labstock/api/borrows.php -H "Content-Type: application/json" -d '{"id":5}'
```

ตัวอย่างคำตอบ

5 บรรทัด

```
{"ok":true,"data":{"id":2,"username":"somchai","name":"สมชาย ใจดี","student_id":"674659001","role":"student"}}
```

```
{"ok":false,"error":"ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง"} HTTP 401
```

```
{"ok":false,"error":"จำนวนคงเหลือไม่พอ"} HTTP 400
```

✓ Checkpoint

- index.php แสดง rows: users 3, items 8, borrows 4
- items.php ได้ JSON 8 ขึ้น ภาษาไทยอ่านได้ (ไม่เป็น ???)
- login ด้วย somchai/1234 ได้ ok:true · รหัสผิดได้ ok:false
- ยืม Arduino 1 ขึ้นแล้วเปิด items.php?id=1 เห็น available ลดลง 1 · คืนแล้วกลับมาเท่าเดิม
- ยืม DHT22 (available 0) ต้องได้ error "จำนวนคงเหลือไม่พอ"

สำหรับผู้สอน

- ให้ดูใน phpMyAdmin คู่กันทุกครั้งที่ curl เพื่อเชื่อม "คำขอ HTTP" กับ "แถวในตาราง"
- ถ้าเห็น HTML แทน JSON แปลว่า PHP error ให้เปิด URL นั้นในเบราว์เซอร์จะเห็นข้อความ error ชัดกว่า
- ถามชวนคิด: ถ้าไม่มี transaction แล้วไปดับหลัง UPDATE items แต่ก่อน INSERT borrows จะเกิดอะไร
- ข้อจำกัดที่ต้องบอกตรง ๆ: API นี้ยังไม่มี token ใครก็ส่ง user_id ของคนอื่นได้ เป็นเรื่องของเฟส 3

ฝั่ง Flutter: ApiService และการแปลง JSON

เวลาโดยประมาณ 60 นาที

เป้าหมาย: แอปมีชั้น ApiService ที่เรียก API ได้ทุก endpoint และโมเดลแปลง JSON ↔ object ได้

ตอนนี้เราจะเพิ่ม "ชั้นเครือข่าย" เข้าไปโดยยังไม่แตะหน้าจอ ไฟล์ใหม่ 2 ไฟล์ + แก๊โมเดล 3 ไฟล์เล็กน้อย + ตั้งค่าสิทธิ์อินเทอร์เน็ต

ความรู้ใหม่ในขั้นนี้

- package `http` : `http.get / post / put` คืน `Future<Response>`
- `jsonEncode / jsonDecode` และ `utf8.decode(res.bodyBytes)` (สำคัญกับภาษาไทย)
- factory constructor `fromJson` และเมธอด `toJson`
- Exception ที่กำหนดเอง (`ApiException`) เพื่อส่งข้อความผิดพลาดจากเซิร์ฟเวอร์ถึงหน้าจอ
- `timeout` และการดัก `ClientException`
- Android Emulator ใช้ 10.0.2.2 แทน localhost · Android 9+ และ iOS บล็อก http ธรรมดา ต้องเปิดใน manifest/plist

1) ติดตั้ง package

```
flutter pub add http
```

2) lib/data/api_config.dart (ไฟล์ใหม่)

lib/data/api_config.dart

ไฟล์ใหม่ · คัดลอกทั้งไฟล์

```
import 'package:flutter/foundation.dart';

/// =====
/// ที่อยู่ของ API - แก๊ค้านี้ค่าเดียว
/// =====
///
/// XAMPP ในเครื่องตัวเอง (โฟลเดอร์ htdocs/labstock/api):
/// 'http://localhost/labstock/api'
///
/// มือถือจริงต่อ Wi-Fi วงเดียวกับคอม (ดู IP จาก ipconfig / ifconfig):
/// 'http://192.168.1.23/labstock/api'
///
/// โสสด์จริง (เช่น AlwaysData):
/// 'https://ชื่อบัญชี.alwaysdata.net/api'
///
```

```

const String kApiBase = 'http://localhost/labstock/api';

/// Android Emulator มองเครื่องคอมของเราเป็น 10.0.2.2 ไม่ใช่ localhost
/// ฟังก์ชันนี้แปลงให้อัดโนมิตี ส่วนแพลตฟอร์มอื่นใช้ค่าตรง ๆ
String get apiBase {
  if (!kIsWeb && defaultTargetPlatform == TargetPlatform.android) {
    return kApiBase.replaceFirst('localhost', '10.0.2.2');
  }
  return kApiBase;
}

```

3) lib/data/api_service.dart (ไฟล์ใหม่)

```

lib/data/api_service.dart ไฟล์ใหม่ · คัดลอกทั้งไฟล์

import 'dart:async';
import 'dart:convert';

import 'package:http/http.dart' as http;

import '../models/borrow.dart';
import '../models/equipment.dart';
import '../models/user.dart';
import 'api_config.dart';

/// ข้อผิดพลาดจาก API - ข้อความพร้อมแสดงให้ผู้ใช้เห็นได้เลย
class ApiException implements Exception {
  final String message;
  final int status;
  ApiException(this.message, [this.status = 0]);

  @override
  String toString() => message;
}

/// ชั้นเดียวที่คุยกับเซิร์ฟเวอร์ - หน้าจอไม่เรียกไฟล์นี้โดยตรง แต่เรียกผ่าน AppState
class ApiService {
  final http.Client _client;
  ApiService([http.Client? client]) : _client = client ?? http.Client();

  // ----- Auth -----
  Future<AppUser> login(String username, String password) async {
    final data = await _send('POST', 'login.php',
      body: {'username': username, 'password': password});
    return AppUser.fromJson(data);
  }

  Future<AppUser> register({
    required String username,
    required String password,
    required String name,
    required String studentId,
  }) async {
    final data = await _send('POST', 'register.php', body: {
      'username': username,
      'password': password,

```

```

        'name': name,
        'student_id': studentId,
    });
    return AppUser.fromJson(data);
}

// ----- Items -----
Future<List<Equipment>> getItems() async {
    final data = await _send('GET', 'items.php') as List;
    return data.map((e) => Equipment.fromJson(e)).toList();
}

Future<Equipment> createItem(Equipment item) async {
    final data = await _send('POST', 'items.php', body: item.toJson());
    return Equipment.fromJson(data);
}

Future<Equipment> updateItem(Equipment item) async {
    final data = await _send('PUT', 'items.php', body: item.toJson());
    return Equipment.fromJson(data);
}

// ----- Borrows -----
Future<List<Borrow>> getBorrows(String userId) async {
    final data =
        await _send('GET', 'borrows.php', query: {'user_id': userId}) as List;
    return data.map((e) => Borrow.fromJson(e)).toList();
}

Future<Borrow> borrow({
    required String userId,
    required String itemId,
    required int quantity,
    required DateTime dueDate,
}) async {
    final data = await _send('POST', 'borrows.php', body: {
        'user_id': userId,
        'item_id': itemId,
        'quantity': quantity,
        'due_date': dueDate.toIso8601String().substring(0, 10), // YYYY-MM-DD
    });
    return Borrow.fromJson(data);
}

Future<Borrow> returnItem(String borrowId) async {
    final data = await _send('PUT', 'borrows.php', body: {'id': borrowId});
    return Borrow.fromJson(data);
}

// ----- ตัวส่งค่าขอกกลาง -----
/// ส่งค่าขอ อ่าน JSON {"ok":..., "data":..., "error":...} แล้วคืน data
/// ถ้า ok=false หรือเชื่อมต่อไม่ได้ → โยน ApiException
Future<dynamic> _send(
    String method,
    String path, {
    Map<String, String>? query,
    Object? body,

```

```

}) async {
  final uri = Uri.parse('$apiBase/$path').replace(queryParameters: query);
  final headers = {'Content-Type': 'application/json'};
  final encoded = body == null ? null : jsonEncode(body);

  http.Response res;
  try {
    final future = switch (method) {
      'GET' => _client.get(uri, headers: headers),
      'POST' => _client.post(uri, headers: headers, body: encoded),
      'PUT' => _client.put(uri, headers: headers, body: encoded),
      _ => throw ArgumentError('method $method'),
    };
    res = await future.timeout(const Duration(seconds: 10));
  } on TimeoutException {
    throw ApiException('เซิร์ฟเวอร์ไม่ตอบสนอง (timeout)\n$uri');
  } on http.ClientException catch (e) {
    throw ApiException('เชื่อมต่อเซิร์ฟเวอร์ไม่ได้: ${e.message}\n$uri');
  }

  Map<String, dynamic> json;
  try {
    json = jsonDecode(utf8.decode(res.bodyBytes)) as Map<String, dynamic>;
  } catch (_) {
    // เซิร์ฟเวอร์ตอบเป็น HTML (เช่น PHP error หรือหน้าตรวจเบราว์เซอร์ของโฮสต์ฟรี)
    throw ApiException(
      'เซิร์ฟเวอร์ตอบไม่ใช่ JSON (HTTP ${res.statusCode})\n$uri');
  }

  if (json['ok'] != true) {
    throw ApiException(
      json['error']?.toString() ?? 'เกิดข้อผิดพลาด', res.statusCode);
  }
  return json['data'];
}
}

```

ทุกเมธอดเรียก `_send` ตัวเดียว ซึ่งจัดการ URL, header, timeout, แปลง JSON และเช็ค `ok` ให้ หน้าจอจึงไม่ต้องรู้เรื่อง HTTP เลย

4) แก้มเดล 3 ไฟล์ — เพิ่ม fromJson / toJson

```

lib/models/user.dart ไฟล์เดิม · แก้ +8 / -0 บรรทัด

@@ -16,3 +16,11 @@ class AppUser {
  bool get isAdmin => role == UserRole.admin;
  String get roleLabel => isAdmin ? 'แอดมิน' : 'นักศึกษา';
+
+  /// แปลงจาก JSON ที่ API ส่งมา (ตาราง users)
+  factory AppUser.fromJson(Map<String, dynamic> json) => AppUser(
+    id: '${json['id']}',
+    name: json['name'] as String,
+    studentId: json['student_id'] as String,
+    role: json['role'] == 'admin' ? UserRole.admin : UserRole.student,

```

```
+ );
}
```

lib/models/equipment.dart

ไฟล์เดิม · แก้ +22 / -1 บรรทัด

```
@@ -1,5 +1,5 @@
import 'package:flutter/material.dart';

-/// หมวดหมู่อุปกรณ์
+/// หมวดหมู่อุปกรณ์ - ชื่อ (board, sensor, ...) ตรงกับคอลัมน์ category ในฐานข้อมูล
enum EquipmentCategory {
  board('บอร์ด', Icons.developer_board),
@@ -34,3 +34,24 @@ class Equipment {

  bool get isOutOfStock => available <= 0;
+
+ /// แปลงจาก JSON ที่ API ส่งมา (ตาราง items)
+ factory Equipment.fromJson(Map<String, dynamic> json) => Equipment(
+   id: '${json['id']}',
+   name: json['name'] as String,
+   category: EquipmentCategory.values.byName(json['category'] as String),
+   total: json['total'] as int,
+   available: json['available'] as int,
+   description: (json['description'] ?? '') as String,
+   imagePath: json['image_url'] as String?,
+ );
+
+ /// แปลงเป็น JSON เพื่อส่งไปเพิ่ม/แก้ไข (available ให้เซิร์ฟเวอร์คำนวณเอง)
+ Map<String, dynamic> toJson() => {
+   'id': id,
+   'name': name,
+   'category': category.name,
+   'total': total,
+   'description': description,
+   'image_url': imagePath,
+ };
}
```

lib/models/borrow.dart

ไฟล์เดิม · แก้ +13 / -0 บรรทัด

```
@@ -30,3 +30,16 @@ class Borrow {
  bool get isOverdue => !isReturned && daysLeft < 0;
  bool get isDueSoon => !isReturned && daysLeft >= 0 && daysLeft <= 3;
+
+ /// แปลงจาก JSON ที่ API ส่งมา (ตาราง borrows JOIN items)
+ factory Borrow.fromJson(Map<String, dynamic> json) => Borrow(
+   id: '${json['id']}',
+   equipmentId: '${json['item_id']}',
+   equipmentName: json['item_name'] as String,
+   quantity: json['quantity'] as int,
+   borrowDate: DateTime.parse(json['borrow_date'] as String),
+   dueDate: DateTime.parse(json['due_date'] as String),
+   returnDate: json['return_date'] == null
+     ? null
+     : DateTime.parse(json['return_date'] as String),
+ );
}
```

```
+ );  
}
```

ข้อสังเกต: id จากเซิร์ฟเวอร์เป็นตัวเลข แต่โมเดลของเราใช้ String จึงแปลงด้วย `'${json['id']}'` เพื่อไม่ต้องแก้ทั้งแอป

5) อนุญาตให้แอปใช้อินเทอร์เน็ตแบบ http

android/app/src/main/AndroidManifest.xml ไฟล์เดิม · แก้ +3 / -0 บรรทัด

```
@@ -1,4 +1,7 @@  
<manifest xmlns:android="http://schemas.android.com/apk/res/android">  
+ <!-- เฟส 2: ให้แอปใช้อินเทอร์เน็ต -->  
+ <uses-permission android:name="android.permission.INTERNET" />  
  <application  
+   android:usesCleartextTraffic="true"  
     android:label="labstock"  
     android:name="${applicationName}"
```

ios/Runner/Info.plist ไฟล์เดิม · แก้ +5 / -0 บรรทัด

```
@@ -27,4 +27,9 @@  
  <key>LSRequiresIPhoneOS</key>  
  <true/>  
+  <key>NSAppTransportSecurity</key>  
+  <dict>  
+    <key>NSAllowsArbitraryLoads</key>  
+    <true/>  
+  </dict>  
  <key>NSCameraUsageDescription</key>  
  <string>ใช้กล้องเพื่อถ่ายรูปอุปกรณ์</string>
```

✓ Checkpoint

- `flutter analyze` → No issues found! (แอปยังทำงานเหมือนเดิม เพราะยังไม่ได้ใช้ ApiService)
- อธิบายได้ว่า `apiBase` จะเป็นค่าอะไรบน Android Emulator, iOS Simulator และมีมือถือจริง
- อธิบายได้ว่าทำไมต้องใช้ `utf8.decode(res.bodyBytes)` แทน `res.body`

สำหรับผู้สอน

- ให้เปิด `items.php` ในเบราว์เซอร์ค้างไว้ข้าง ๆ โค้ด `fromJson` แล้วชี้ทีละ key ว่าไปลง field ไหน
- ย้ำเรื่อง URL 3 แบบ (`localhost` / `10.0.2.2` / IP วง LAN) เป็นจุดที่นักศึกษาติดมากที่สุดในคาบนี้

เปลี่ยน AppState ไปใช้ API และปรับหน้าจอ

เวลาโดยประมาณ 80 นาที

เป้าหมาย: แอปทำงานกับข้อมูลจริงใน MySQL ครบทุก flow: login, รายการ, ยืม, คืน, เพิ่ม/แก้ไข, สมัครสมาชิก

นี่คือขั้นที่เห็นผล เปลี่ยน **AppState** จาก "เก็บ list ในหน่วยความจำ" เป็น "เรียก ApiService แล้วเก็บผลลัพธ์" เมธอดชื่อเดิมทั้งหมดแต่กลายเป็น **Future** หน้าจอจึงต้องเติม **await** และดัก **ApiException**

ความรู้ใหม่ในขั้นนี้

- async/await ในเมธอดของ StatefulWidget และการเช็ค **mounted** หลัง await
- แพตเทิร์น loading: **setState(() => _busy = true)** → try/await → finally reset
- แสดง 3 สถานะ: กำลังโหลด / ผิดพลาด+ลองใหม่ / ข้อมูล
- **RefreshIndicator** ดึงลงเพื่อโหลดใหม่
- หลังเขียนข้อมูลให้ **refresh()** ดึงจากเซิร์ฟเวอร์ใหม่ทั้งหมด = เซิร์ฟเวอร์เป็น single source of truth
- ทดสอบโดยไม่ต้องมีเซิร์ฟเวอร์ด้วยการ inject **FakeApiService**

1) lib/data/app_state.dart (แทนที่ทั้งไฟล์)

lib/data/app_state.dart

ไฟล์ใหม่ · คัดลอกทั้งไฟล์

```
import 'package:flutter/foundation.dart';

import '../models/borrow.dart';
import '../models/equipment.dart';
import '../models/user.dart';
import 'api_service.dart';

/// สถานะรวมของแอป - เฟส 2: ข้อมูลทุกอย่างมาจาก API/MySQL ผ่าน [ApiService]
/// หน้าจอยังเรียกเมธอดชื่อเดิม แต่ตอนนี้เป็น Future (ต้อง await)
class AppState extends ChangeNotifier {
  AppState._();
  static final AppState instance = AppState._();

  /// ตัวคุยกับเซิร์ฟเวอร์ - เปลี่ยนเป็นตัวปลอมได้ตอนทดสอบ
  ApiService api = ApiService();

  AppUser? _currentUser;
  List<Equipment> _equipment = [];
  List<Borrow> _borrows = [];

  bool isLoading = false;
  String? error;
```

```

AppUser? get currentUser => _currentUser;
bool get isAdmin => _currentUser?.isAdmin ?? false;
List<Equipment> get equipment => List.unmodifiable(_equipment);
List<Borrow> get borrows => List.unmodifiable(_borrows);

List<Borrow> get activeBorrows =>
    _borrows.where((b) => !b.isReturned).toList()
        ..sort((a, b) => a.dueDate.compareTo(b.dueDate));

List<Borrow> get borrowHistory =>
    _borrows.where((b) => b.isReturned).toList()
        ..sort((a, b) => b.returnDate!.compareTo(a.returnDate!));

// ----- Auth -----
/// เข้าสู่ระบบผ่าน API แล้วโหลดข้อมูลทั้งหมด (โยน ApiException ถ้าไม่สำเร็จ)
Future<void> login(String username, String password) async {
    _currentUser = await api.login(username.trim(), password);
    notifyListeners();
    await refresh();
}

Future<void> register({
    required String username,
    required String password,
    required String name,
    required String studentId,
}) async {
    await api.register(
        username: username.trim(),
        password: password,
        name: name.trim(),
        studentId: studentId.trim(),
    );
}

void logout() {
    _currentUser = null;
    _equipment = [];
    _borrows = [];
    notifyListeners();
}

// ----- โหลดข้อมูลจากเซิร์ฟเวอร์ -----
/// ดึงอุปกรณ์ + รายการยืมของผู้ใช้ปัจจุบันใหม่ทั้งหมด (ใช้หลังทุกการเปลี่ยนแปลง)
Future<void> refresh() async {
    isLoading = true;
    error = null;
    notifyListeners();
    try {
        _equipment = await api.getItems();
        final user = _currentUser;
        _borrows = user == null ? [] : await api.getBorrows(user.id);
    } on ApiException catch (e) {
        error = e.message;
    } finally {
        isLoading = false;
        notifyListeners();
    }
}

```

```

    }
  }

  // ----- Equipment -----
  Equipment? findEquipment(String id) {
    for (final e in _equipment) {
      if (e.id == id) return e;
    }
    return null;
  }

  Future<void> addEquipment(Equipment item) async {
    await api.createItem(item);
    await refresh();
  }

  Future<void> updateEquipment(Equipment item) async {
    await api.updateItem(item);
    await refresh();
  }

  // ----- Borrow / Return -----
  Future<void> borrow(Equipment item, int quantity, DateTime dueDate) async {
    final user = _currentUser;
    if (user == null) throw ApiException('กรุณาเข้าสู่ระบบก่อน');
    await api.borrow(
      userId: user.id,
      itemId: item.id,
      quantity: quantity,
      dueDate: dueDate,
    );
    await refresh();
  }

  Future<void> returnItem(Borrow b) async {
    await api.returnItem(b.id);
    await refresh();
  }
}

```

เทียบกับเฟส 1: `login` ไม่ได้ใช้ชื่อขึ้นต้นด้วย `admin` แล้ว แต่ให้เซิร์ฟเวอร์ตัดสินใจ · ทุกเมธอดที่เขียนข้อมูลจบด้วย

`await refresh()`

2) แก้น้ำจ่อ — ดู diff แล้วแก้ตามเฉพาะบรรทัดสีเขียว/แดง

`login_screen.dart` — เพิ่ม `_busy`, เปลี่ยน `_login` เป็น `async`, ปุ่มแสดงวงกลมหมุนตอนรอ

lib/screens/login_screen.dart

ไฟล์เดิม · แก้ +27 / -9 บรรทัด

```

@@ -1,4 +1,5 @@
import 'package:flutter/material.dart';

+import '../data/api_service.dart';
import '../data/app_state.dart';
import '../theme.dart';

```

```

@@ -18,4 +19,5 @@ class _LoginScreenState extends State<LoginScreen> {
    final _passCtrl = TextEditingController();
    bool _obscure = true;
+   bool _busy = false; // กำลังรอเซิร์ฟเวอร์

    @override
@@ -26,7 +28,6 @@ class _LoginScreenState extends State<LoginScreen> {
    }

-   void _login() {
-       final ok = AppState.instance.login(_userCtrl.text, _passCtrl.text);
-       if (!ok) {
+   Future<void> _login() async {
+       if (_userCtrl.text.trim().isEmpty || _passCtrl.text.isEmpty) {
           ScaffoldMessenger.of(context).showSnackBar(
               const SnackBar(content: Text('กรุณกรอกอีเมล/ชื่อผู้ใช้ และรหัสผ่าน'))),
@@ -34,7 +35,18 @@ class _LoginScreenState extends State<LoginScreen> {
           return;
       }
-       Navigator.of(context).pushReplacement(
-           MaterialPageRoute(builder: (_) => const HomeShell()),
-       );
+       setState(() => _busy = true);
+       try {
+           await AppState.instance.login(_userCtrl.text, _passCtrl.text);
+           if (!mounted) return;
+           Navigator.of(context).pushReplacement(
+               MaterialPageRoute(builder: (_) => const HomeShell()),
+           );
+       } on ApiException catch (e) {
+           if (!mounted) return;
+           ScaffoldMessenger.of(context)
+               .showSnackBar(SnackBar(content: Text(e.message)));
+       } finally {
+           if (mounted) setState(() => _busy = false);
+       }
    }

@@ -105,10 +117,16 @@ class _LoginScreenState extends State<LoginScreen> {
    const SizedBox(height: 22),
    ElevatedButton(
-       onPressed: _login,
-       child: const Text('เข้าสู่ระบบ'),
+       onPressed: _busy ? null : _login,
+       child: _busy
+           ? const SizedBox(
+               width: 22,
+               height: 22,
+               child: CircularProgressIndicator(
+                   strokeWidth: 2, color: Colors.white))
+           : const Text('เข้าสู่ระบบ'),
    ),
    const SizedBox(height: 8),
    Text(
-       'ทดลอง: ชื่อผู้ใช้เริ่มต้นด้วย "admin" = แอดมิน · อื่น ๆ = นักศึกษา',
+       'ทดลอง: admin / 1234 (แอดมิน) · somchai / 1234 (นักศึกษา)',
    ),

```

```
textAlign: TextAlign.center,  
style: text.bodySmall?.copyWith(color: Colors.grey),
```

equipment_list_screen.dart — เพิ่มปุ่มโหลดใหม่, RefreshIndicator และ _buildGrid ที่แสดงสถานะโหลด/ผิดพลาด

lib/screens/equipment_list_screen.dart

ไฟล์เดิม · แก้ +47 / -14 บรรทัด

```
@@ -40,4 +40,9 @@ class _EquipmentListScreenState extends State<EquipmentListScreen>  
{  
    title: const Text('LabStock'),  
    actions: [  
+      IconButton(  
+        icon: const Icon(Icons.refresh),  
+        tooltip: 'โหลดใหม่',  
+        onPressed: state.isLoading ? null : state.refresh,  
+      ),  
      IconButton(  
        icon: const Icon(Icons.notifications_outlined),  
@@ -82,18 +87,9 @@ class _EquipmentListScreenState extends State<EquipmentListScreen  
> {  
    // ----- ส่วนกลาง: รายการ -----  
    Expanded(  
-      child: items.isEmpty  
-        ? const Center(child: Text('ไม่พบอุปกรณ์ที่ค้นหา'))  
-        : GridView.builder(  
-          padding: const EdgeInsets.fromLTRB(16, 4, 16, 88),  
-          gridDelegate:  
-            const SliverGridDelegateWithMaxCrossAxisExtent(  
-              maxCrossAxisExtent: 220,  
-              mainAxisSpacing: 12,  
-              crossAxisSpacing: 12,  
-              childAspectRatio: 0.82,  
-            ),  
-          itemCount: items.length,  
-          itemBuilder: (_, i) => _EquipmentCard(item: items[i]),  
-        ),  
+      // ดึงลงเพื่อโหลดใหม่จากเซิร์ฟเวอร์  
+      child: RefreshIndicator(  
+        onRefresh: state.refresh,  
+        child: _buildGrid(state, items),  
+      ),  
    ),  
  ],  
@@ -115,4 +111,41 @@ class _EquipmentListScreenState extends State<EquipmentListScreen> {  
  }  
  
+ /// เลือกว่าจะแสดงอะไร: กำลังโหลด / ผิดพลาด / ไม่พบ / กริดปกติ  
+ Widget _buildGrid(AppState state, List<Equipment> items) {  
+   if (state.isLoading && state.equipment.isEmpty) {  
+     return const Center(child: CircularProgressIndicator());  
+   }  
+   if (state.error != null && state.equipment.isEmpty) {  
+     return ListView(  
+       padding: const EdgeInsets.all(32),
```

```

+     children: [
+       Icon(Icons.cloud_off, size: 56, color: Colors.grey[400]),
+       const SizedBox(height: 12),
+       Text(state.error!, textAlign: TextAlign.center),
+       TextButton(onPressed: state.refresh, child: const Text('ลองใหม่')),
+     ],
+   );
+ }
+ if (items.isEmpty) {
+   return ListView(
+     children: const [
+       SizedBox(height: 80),
+       Center(child: Text('ไม่พบอุปกรณ์ที่ค้นหา')),
+     ],
+   );
+ }
+ return GridView.builder(
+   padding: const EdgeInsets.fromLTRB(16, 4, 16, 88),
+   gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(
+     maxCrossAxisExtent: 220,
+     mainAxisSpacing: 12,
+     crossAxisSpacing: 12,
+     childAspectRatio: 0.82,
+   ),
+   itemCount: items.length,
+   itemBuilder: (_, i) => _EquipmentCard(item: items[i]),
+ );
+ }
+

```

```

Widget _chip(String label, EquipmentCategory? cat) {
  final selected = _category == cat;

```

equipment_detail_screen.dart — await borrow และดัก ApiException

```

lib/screens/equipment_detail_screen.dart ไฟล์เดิม · แก้ +12 / -5 บรรทัด

@@ -1,4 +1,5 @@
import 'package:flutter/material.dart';

+import '../data/api_service.dart';
import '../data/app_state.dart';
import '../models/equipment.dart';
@@ -97,14 +98,20 @@ class EquipmentDetailScreen extends StatelessWidget {
  if (result == null || !context.mounted) return;

-   final ok = AppState.instance.borrow(item, result.quantity, result.dueDate);
+   try {
+     await AppState.instance.borrow(item, result.quantity, result.dueDate);
+   } on ApiException catch (e) {
+     if (!context.mounted) return;
+     ScaffoldMessenger.of(context)
+       .showSnackBar(SnackBar(content: Text(e.message)));
+     return;
+   }
  if (!context.mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(

```

```

        SnackBar(
-         content: Text(ok
-             ? 'ยืม ${item.name} x${result.quantity} สำเร็จ · คืน ${thaiShortDate(result.dueDate)}'
-             : 'ยืมไม่สำเร็จ'),
+         content: Text(
+             'ยืม ${item.name} x${result.quantity} สำเร็จ · คืน ${thaiShortDate(result.dueDate)}'),
        ),
    );
-     if (ok) Navigator.of(context).pop(); // กลับหน้ารายการ
+     Navigator.of(context).pop(); // กลับหน้ารายการ
  }
}

```

my_borrows_screen.dart — RefreshIndicator และ await returnItem

```

lib/screens/my_borrows_screen.dart ไฟล์เดิม · แก้ +17 / -5 บรรทัด

@@ -1,4 +1,5 @@
import 'package:flutter/material.dart';

+import '../data/api_service.dart';
import '../data/app_state.dart';
import '../models/borrow.dart';
@@ -64,8 +65,11 @@ class _BorrowList extends StatelessWidget {
  );
}

-   return ListView.builder(
-     padding: const EdgeInsets.all(16),
-     itemCount: borrows.length,
-     itemBuilder: (_, i) => _BorrowCard(borrow: borrows[i]),
+   return RefreshIndicator(
+     onRefresh: AppState.instance.refresh,
+     child: ListView.builder(
+       padding: const EdgeInsets.all(16),
+       itemCount: borrows.length,
+       itemBuilder: (_, i) => _BorrowCard(borrow: borrows[i]),
+     ),
  );
}

@@ -95,5 +99,13 @@ class _BorrowCard extends StatelessWidget {
  );
  if (ok != true || !context.mounted) return;
-   AppState.instance.returnItem(borrow);
+   try {
+     await AppState.instance.returnItem(borrow);
+   } on ApiException catch (e) {
+     if (!context.mounted) return;
+     ScaffoldMessenger.of(context)
+       .showSnackBar(SnackBar(content: Text(e.message)));
+     return;
+   }
+   if (!context.mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('คืน ${borrow.equipmentName} สำเร็จ')),
  );
}

```

lib/screens/equipment_form_screen.dart

ไฟล์เดิม · แก้ +36 / -23 บรรทัด

```

@@ -6,4 +6,5 @@ import 'package:flutter/services.dart';
import 'package:image_picker/image_picker.dart';

+import '../data/api_service.dart';
import '../data/app_state.dart';
import '../models/equipment.dart';
@@ -30,4 +31,5 @@ class _EquipmentFormScreenState extends State<EquipmentFormScreen>
{
    widget.existing?.category ?? EquipmentCategory.board;
    late String? _imagePath = widget.existing?.imagePath;
+    bool _saving = false;

    bool get _isEdit => widget.existing != null;
@@ -73,32 +75,43 @@ class _EquipmentFormScreenState extends State<EquipmentFormScreen> {

- void _save() {
+ Future<void> _save() async {
    if (!_formKey.currentState!.validate()) return;
    final state = AppState.instance;
    final total = int.parse(_totalCtrl.text.trim());

-   if (_isEdit) {
-     final item = widget.existing!;
-     final borrowed = item.total - item.available;
-     item
-       ..name = _nameCtrl.text.trim()
-       ..category = _category
-       ..total = total
-       ..available = (total - borrowed).clamp(0, total)
-       ..description = _descCtrl.text.trim()
-       ..imagePath = _imagePath;
-     state.updateEquipment(item);
-   } else {
-     state.addEquipment(Equipment(
-       id: 'e${DateTime.now().millisecondsSinceEpoch}',
-       name: _nameCtrl.text.trim(),
-       category: _category,
-       total: total,
-       available: total,
-       description: _descCtrl.text.trim(),
-       imagePath: _imagePath,
-     ));
+   setState(() => _saving = true);
+   try {
+     if (_isEdit) {
+       final item = widget.existing!;
+       final borrowed = item.total - item.available;
+       item
+         ..name = _nameCtrl.text.trim()
+         ..category = _category
+         ..total = total

```

```

+         ..available = (total - borrowed).clamp(0, total)
+         ..description = _descCtrl.text.trim()
+         ..imagePath = _imagePath;
+         await state.updateEquipment(item); // เซิร์ฟเวอร์คำนวณ available เองอีกครั้ง
+     } else {
+         await state.addEquipment(Equipment(
+             id: '0', // เซิร์ฟเวอร์เป็นคนกำหนด id จริง
+             name: _nameCtrl.text.trim(),
+             category: _category,
+             total: total,
+             available: total,
+             description: _descCtrl.text.trim(),
+             imagePath: _imagePath,
+         ));
+     }
+ } on ApiException catch (e) {
+     if (!mounted) return;
+     ScaffoldMessenger.of(context)
+         .showSnackBar(SnackBar(content: Text(e.message)));
+     return;
+ } finally {
+     if (mounted) setState(() => _saving = false);
+ }

+ if (!mounted) return;
+ ScaffoldMessenger.of(context).showSnackBar(
+     SnackBar(content: Text(_isEdit ? 'บันทึกการแก้ไขแล้ว' : 'เพิ่มอุปกรณ์แล้ว')),
@@ -204,5 +217,5 @@ class _EquipmentFormScreenState extends State<EquipmentFormScreen> {
+         // ----- ส่วนล่าง: บันทึก -----
+         ElevatedButton.icon(
-             onPressed: _save,
+             onPressed: _saving ? null : _save,
+             icon: const Icon(Icons.save_outlined),
+             label: const Text('บันทึก'),

```

register_screen.dart — เฟส 1 ยังไม่มี controller เลย จึงแทนที่ทั้งไฟล์ง่ายกว่า

lib/screens/register_screen.dart

ไฟล์ใหม่ · คัดลอกทั้งไฟล์

```

import 'package:flutter/material.dart';

import '../data/api_service.dart';
import '../data/app_state.dart';

/// หน้าสมัครสมาชิก — เฟส 2: บันทึกผู้ใช้ใหม่ลงฐานข้อมูลผ่าน API
class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameCtrl = TextEditingController();

```

```

final _idCtrl = TextEditingController();
final _userCtrl = TextEditingController();
final _passCtrl = TextEditingController();
bool _obscure = true;
bool _busy = false;

@override
void dispose() {
  _nameCtrl.dispose();
  _idCtrl.dispose();
  _userCtrl.dispose();
  _passCtrl.dispose();
  super.dispose();
}

Future<void> _submit() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() => _busy = true);
  try {
    await AppState.instance.register(
      username: _userCtrl.text,
      password: _passCtrl.text,
      name: _nameCtrl.text,
      studentId: _idCtrl.text,
    );
  } on ApiException catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(context)
      .showSnackBar(SnackBar(content: Text(e.message)));
    return;
  } finally {
    if (mounted) setState(() => _busy = false);
  }
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('สมัครสมาชิกสำเร็จ - กรุณาเข้าสู่ระบบ')),
  );
  Navigator.of(context).pop();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('สมัครสมาชิก')),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(24),
      child: Form(
        key: _formKey,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            TextFormField(
              controller: _nameCtrl,
              decoration: const InputDecoration(
                labelText: 'ชื่อ-นามสกุล',
                prefixIcon: Icon(Icons.badge_outlined),
              ),
            ),

```

```

        validator: (v) =>
          (v == null || v.trim().isEmpty) ? 'กรุณากรอกชื่อ' : null,
      ),
      const SizedBox(height: 14),
      TextFormField(
        controller: _idCtrl,
        keyboardType: TextInputType.number,
        decoration: const InputDecoration(
          labelText: 'รหัสนักศึกษา',
          prefixIcon: Icon(Icons.numbers),
        ),
      ),
      validator: (v) => (v == null || v.trim().length < 9)
        ? 'รหัสนักศึกษาต้องมีอย่างน้อย 9 หลัก'
        : null,
    ),
    const SizedBox(height: 14),
    TextFormField(
      controller: _userCtrl,
      keyboardType: TextInputType.emailAddress,
      decoration: const InputDecoration(
        labelText: 'อีเมล/ชื่อผู้ใช้',
        prefixIcon: Icon(Icons.person_outline),
      ),
    ),
    validator: (v) => (v == null || v.trim().length < 3)
      ? 'ชื่อผู้ใช้ต้องมีอย่างน้อย 3 ตัวอักษร'
      : null,
  ),
  const SizedBox(height: 14),
  TextFormField(
    controller: _passCtrl,
    obscureText: _obscure,
    decoration: InputDecoration(
      labelText: 'รหัสผ่าน',
      prefixIcon: const Icon(Icons.lock_outline),
      suffixIcon: IconButton(
        icon: Icon(_obscure
          ? Icons.visibility_outlined
          : Icons.visibility_off_outlined),
        onPressed: () => setState(() => _obscure = !_obscure),
      ),
    ),
  ),
  validator: (v) => (v == null || v.length < 6)
    ? 'รหัสผ่านต้องมีอย่างน้อย 6 ตัวอักษร'
    : null,
),
const SizedBox(height: 24),
ElevatedButton(
  onPressed: _busy ? null : _submit,
  child: const Text('สมัครสมาชิก'),
),
TextButton(
  onPressed: () => Navigator.of(context).pop(),
  child: const Text('มีบัญชีแล้ว? เข้าสู่ระบบ'),
),
],
),
),
),

```

```

    ),
  );
}
}

```

widgets/equipment_image.dart — รองรับ URL รูปจากเซิร์ฟเวอร์ และไม่พึ่งถ้า path ในเครื่องไม่มีจริง

```

lib/widgets/equipment_image.dart ไฟล์เดิม · แก้ +15 / -5 บรรทัด

@@ -7,5 +7,8 @@ import '../models/equipment.dart';
import '../theme.dart';

-/// รูปอุปกรณ์: ถ้ามีรูปที่เลือกจาก image_picker ให้แสดง ถ้าไม่มีแสดงไอคอนตามหมวด
+/// รูปอุปกรณ์:
+/// - imagePath ขึ้นต้นด้วย http → รูปจากเซิร์ฟเวอร์ (Image.network)
+/// - imagePath เป็นไฟล์ในเครื่องที่มีอยู่จริง → Image.file (จาก image_picker)
+/// - อื่น ๆ → ไอคอนตามหมวด
class EquipmentImage extends StatelessWidget {
  final Equipment item;
  @@ -17,12 +20,19 @@ class EquipmentImage extends StatelessWidget {
    Widget build(BuildContext context) {
      final path = item.imagePath;
      - if (path != null && !kIsWeb) {
      -   return Image.file(File(path), fit: BoxFit.cover);
      - }
      - return Container(
      + final placeholder = Container(
        color: AppColors.sky.withValues(alpha: 0.12),
        alignment: Alignment.center,
        child: Icon(item.category.icon, size: iconSize, color: AppColors.indigo),
      );

      +
      + if (path == null || path.isEmpty) return placeholder;
      + if (path.startsWith('http')) {
      +   return Image.network(path,
      +     fit: BoxFit.cover, errorBuilder: (_, __, ___) => placeholder);
      + }
      + if (!kIsWeb && File(path).existsSync()) {
      +   return Image.file(File(path), fit: BoxFit.cover);
      + }
      + return placeholder;
    }
  }
}

```

3) ทดสอบอัตโนมัติด้วย API ปลอม

```

test/widget_test.dart ไฟล์ใหม่ · คัดลอกทั้งไฟล์

import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:labstock/data/api_service.dart';
import 'package:labstock/data/app_state.dart';
import 'package:labstock/data/mock_data.dart';
import 'package:labstock/main.dart';
import 'package:labstock/models/borrow.dart';

```

```

import 'package:labstock/models/equipment.dart';
import 'package:labstock/models/user.dart';

/// API ปลอมสำหรับทดสอบ - ใช้ข้อมูลจำลองจากเฟส 1 ไม่ต้องมีเซิร์ฟเวอร์จริง
class FakeApiService extends ApiService {
  final List<Equipment> items = MockData.equipment();
  final List<Borrow> borrows = MockData.borrows();

  @override
  Future<AppUser> login(String username, String password) async {
    if (password != '1234') throw ApiException('ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง', 401);
    return username == 'admin' ? MockData.admin : MockData.student;
  }

  @override
  Future<List<Equipment>> getItems() async => items;

  @override
  Future<List<Borrow>> getBorrows(String userId) async => borrows;
}

void main() {
  setUp(() {
    AppState.instance.api = FakeApiService();
    AppState.instance.logout();
  });

  Future<void> login(WidgetTester tester, String user, String pass) async {
    await tester.pumpWidget(const LabStockApp());
    await tester.enterText(find.byType(TextField).first, user);
    await tester.enterText(find.byType(TextField).last, pass);
    await tester.tap(find.widgetWithText(ElevatedButton, 'เข้าสู่ระบบ'));
    await tester.pumpAndSettle();
  }

  testWidgets('login as student then navigate tabs', (tester) async {
    await login(tester, 'somchai', '1234');

    // หน้ารายการ: นักศึกษาไม่เห็นปุ่ม +
    expect(find.text('Arduino UNO R3'), findsOneWidget);
    expect(find.byType(FloatingActionButton), findsNothing);
    expect(find.text('ยืมไม่ได้'), findsOneWidget); // DHT22 เหลือ 0

    await tester.tap(find.text('การยืมของฉัน'));
    await tester.pumpAndSettle();
    expect(find.textContaining('อีก 2 วันครบกำหนด'), findsOneWidget);

    await tester.tap(find.text('โปรไฟล์'));
    await tester.pumpAndSettle();
    expect(find.text('สมชาย ใจดี'), findsOneWidget);
    await tester.tap(find.text('ออกจากระบบ'));
    await tester.pumpAndSettle();
    expect(find.text('เข้าสู่ระบบ'), findsOneWidget);
  });

  testWidgets('admin sees add button', (tester) async {
    await login(tester, 'admin', '1234');
  }

```

```

    expect(find.byType(FloatingActionButton), findsOneWidget);
  });

testWidgets('wrong password shows error from API', (tester) async {
  await login(tester, 'somchai', 'wrong');
  expect(find.text('ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง'), findsOneWidget);
  expect(find.byType(FloatingActionButton), findsNothing);
});
}

```

```

flutter analyze
flutter test
flutter run

```

✓ Checkpoint (ต้องเปิด XAMPP ก่อน)

- login somchai/1234 ได้ · รหัสผิดเห็นข้อความจากเซิร์ฟเวอร์ "ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง"
- หน้ารายการโหลดจาก MySQL (ลองแก้ไขอุปกรณ์ใน phpMyAdmin แล้วดึงลงรีเฟรช ต้องเปลี่ยนตาม)
- ยืม Arduino → phpMyAdmin ตาราง borrows มีแถวใหม่ และ items.available ลด
- คืนของ → return_date มีค่า และ available กลับมา
- login admin/1234 → เพิ่มอุปกรณ์ → เห็นในตาราง items
- สมัครสมาชิกใหม่ → login ด้วยบัญชีนั้นได้
- ปิด MySQL แล้วเปิดแอป → เห็นหน้า "เชื่อมต่อเซิร์ฟเวอร์ไม่ได้" พร้อมปุ่มลองใหม่ ไม่ crash
- `flutter test` → All tests passed! (ผ่านแม้ไม่เปิดเซิร์ฟเวอร์ เพราะใช้ FakeApiService)

สำหรับผู้สอน

- แบ่งเป็น 2 รอบ: รอบแรก app_state + login + list (เห็นข้อมูลจริงขึ้นจอ = กำลังใจ) รอบสองที่เหลือ
- ให้จับคู่: คนหนึ่งเปิด phpMyAdmin คนหนึ่งกดแอป แล้วเล่าให้กันฟังว่าตารางเปลี่ยนอย่างไร
- error ที่พบบ่อยสุด: (1) ลืม await เลยไม่เห็น error (2) ใช้ context หลัง await โดยไม่เช็ก mounted (3) URL ผิดแพลตฟอร์ม

นำขึ้นโฮสต์จริง และก้าวต่อไป

เวลาโดยประมาณ 40 นาที

เป้าหมาย: เข้าใจข้อจำกัดของโฮสต์ฟรี นำ API ขึ้นโฮสต์ที่รองรับแอปมือถือได้ และเห็นภาพเฟส 3

คำเตือนสำคัญเรื่อง InfinityFree

InfinityFree (และโฮสต์ฟรีในเครือ iFastNet เช่น Byet, ProFreeHost) มี **ระบบตรวจสอบเบราว์เซอร์** ที่ต้องรัน JavaScript (aes.js) และเก็บคุกกี้ก่อนจึงจะเข้าถึงไฟล์ PHP ได้ แอป Flutter ไม่ใช่เบราว์เซอร์ จึงได้รับหน้า HTML แทน JSON ทุกครั้ง เอกสารของ InfinityFree ระบุตรง ๆ ว่า *"Android or iOS mobile apps cannot connect to your website"* และ *"REST APIs ... are blocked"* ต้องซื้อ premium (iFastNet) จึงจะปลดล็อก จึงไม่แนะนำให้ใช้กับวิชานี้ ในแอปของเรา error จะขึ้นว่า "เซิร์ฟเวอร์ตอบไม่ใช่ JSON"

ทางเลือกที่ทดสอบแนวคิดแล้วว่าเข้ากับแอปมือถือ: **XAMPP ในเครื่อง** (เหมาะกับในห้องเรียน ไม่พึ่งอินเทอร์เน็ต) และโฮสต์ฟรี **AlwaysData** (PHP + MySQL + HTTPS ไม่มีระบบตรวจสอบเบราว์เซอร์) สำหรับส่งงานหรือลองใช้จากมือถือจริงนอกวง LAN

1) ใช้จากมือถือจริงในห้อง (วง Wi-Fi เดียวกับคอม)

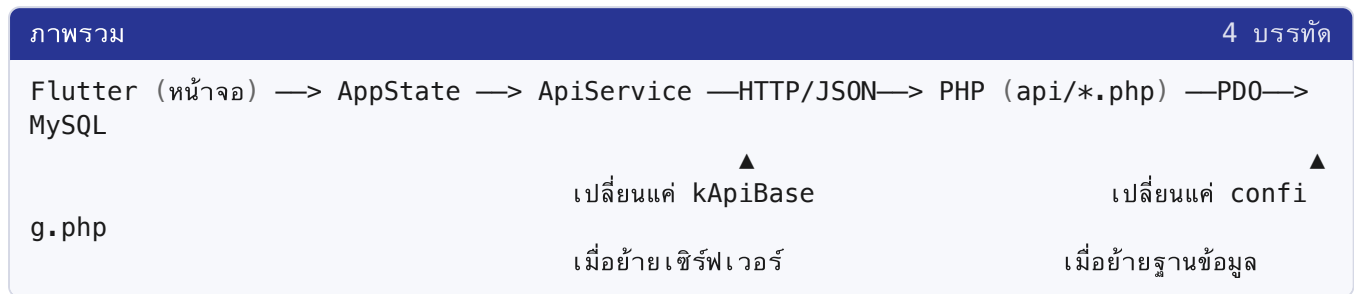
1. หา IP ของคอม: Windows `ipconfig` (IPv4 Address) · Mac `ifconfig en0`
2. Windows: อนุญาต Apache ผ่าน Firewall (ครั้งแรกจะมี popup ตอนสตาร์ท Apache ให้กด Allow)
3. ทดสอบจากเบราว์เซอร์มือถือ: `http://192.168.x.x/labstock/api/index.php`
4. แก้ `kApiBase` ใน `api_config.dart` เป็น IP นั้น แล้วรันแอปบนมือถือ

2) นำขึ้น AlwaysData (ฟรี)

1. สมัครที่ alwaysdata.com เลือกแผนฟรี ตั้งชื่อบัญชี เช่น `labstock-somchai` จะได้เว็บ `https://labstock-somchai.alwaysdata.net`
2. แผงควบคุม → Databases → MySQL → สร้างฐานข้อมูล (ชื่อจะขึ้นต้นด้วยชื่อบัญชี_) และผู้ใช้งานข้อมูล
3. เปิด phpMyAdmin ของ AlwaysData → Import `labstock.sql` โดยลบ 2 บรรทัดแรก (CREATE DATABASE / USE) ออกก่อน เพราะชื่อฐานข้อมูลไม่ใช่ labstock
4. อัปโหลดไฟล์เตอร์ api ไปที่ `www/api` ผ่าน FTP (FileZilla) หรือ File manager
5. แก้ `config.php`: DB_HOST = `mysql-ชื่อบัญชี.alwaysdata.net` DB_NAME / DB_USER / DB_PASS ตามที่สร้าง

- เปิด `https://ชื่อบัญชี.alwaysdata.net/api/index.php` ต้องได้ JSON
- แก้ `kApiBase` ในแอปเป็น URL นั้น (https ไม่ต้องตั้งค่า cleartext ไต ะ)

3) สรุปสถาปัตยกรรมที่ได้



4) เฟส 3 (ทำลาย / โปรเจกต์)

- Token:** login แล้วเซิร์ฟเวอร์ออก token สุ่ม เก็บในตาราง users ทุกคำขอต้องส่ง header Authorization และเซิร์ฟเวอร์ตรวจว่า user_id ตรงกับ token
- จัดการเข้าสู่ระบบ:** เก็บ user ด้วย `shared_preferences` เปิดแอปครั้งถัดไปข้ามหน้า Login
- อัปโหลดรูป:** เพิ่ม upload.php รับไฟล์ (multipart) บันทึกใน htdocs/labstock/uploads แล้วคืน URL → image_url
- หน้าแอดมิน:** GET borrows.php ทั้งหมด (ไม่ระบุ user_id) พร้อมชื่อผู้ยืม แสดงเป็นตารางให้แอดมิน
- แจ้งเตือน:** endpoint นับรายการที่จะครบกำหนดใน 3 วัน แสดงตัวเลขบนกระดิ่ง